Microsoft Teams Phone Enablement — Calling Plans & Operator Connect

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Microsoft Teams Phone Enablement (Calling Plans & Operator Connect) 5

Outcome . . . 5

Approach . . . 5

Why it's low-risk . . . 5

1. Solution Overview — Microsoft Teams Phone Enablement (Calling Plans & Operator Connect) 5

What this solution is . . . 6

The three PSTN options (and why this solution uses two of them) . . . 6

How it works (end to end) . . . 7

Reference design . . . 7

Benefits . . . 8

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 9

The standard deployment model . . . 9

2. Requirements — Microsoft Teams Phone (Calling Plans & Operator Connect) 9

2.1 Teams Phone licensing for every calling user . . . 10

2.2 A chosen PSTN path per number — Calling Plan and/or Operator Connect . . . 10

2.3 Phone numbers available to assign . . . 10

2.4 Emergency addresses and dynamic-location data (LIS) . . . 11

2.5 Emergency-call routing responsibility confirmed (carrier/operator) . . . 11

2.6 Admin roles for voice, numbers, and emergency configuration . . . 12

2.7 Network readiness for voice quality (and dynamic location) . . . 12

2.8 Resource-account service numbers for voice apps (if AAs/CQs are in scope) . . . 12

Global Requirements — Microsoft Teams Solutions 13

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing . . . 13

2. Administrative access (least-privilege Teams roles) . . . 13

3. Microsoft Teams PowerShell + Graph toolchain . . . 14

4. Identity, groups & lifecycle source of truth . . . 14

5. A pilot scope and change control . . . 14

6. Security, compliance & governance stakeholders . . . 14

3. Implementation Guide — Teams Phone (Calling Plans & Operator Connect) 15

Step 1 — Assess readiness and inventory (Assess) . . . 15

Step 2 — Define the pilot scope (Design) . . . 15

Step 3 — Acquire numbers (Design/Configure) . . . 16

Step 4 — Register emergency addresses and dynamic locations (Configure — do this BEFORE assigning numbers) . . . 16
Step 5 — Build the calling policy, dial plan, and emergency calling policy (Configure) . . . 16
Step 6 — Dry-run the pilot enablement (Pilot — no changes yet) . . . 17
Step 7 — Enable the pilot (Pilot) . . . 17
Step 8 — Build auto attendants and call queues (Configure/Pilot) . . . 18
Step 9 — Validate the pilot (Validate) . . . 18
Step 10 — Expand under change control (Expand) . . . 18
5. Testing & Validation — Teams Phone (Calling Plans & Operator Connect) 19
Test 1 — User is enterprise-voice-enabled with the correct number . . . 19
Test 2 — Voice policies and dial plan are assigned . . . 19
Test 3 — Emergency location is attached . . . 20
Test 4 — Spare inventory and no double-assignment . . . 20
Test 5 — Outbound PSTN call . . . 20
Test 6 — Inbound PSTN call . . . 20
Test 7 — Emergency call (coordinated, live) — MANDATORY GATE . . . 21
Test 8 — Auto attendant and call queue . . . 21
6. Troubleshooting — Teams Phone (Calling Plans & Operator Connect) 22
A number won't assign (`Set-CsPhoneNumberAssignment` errors) . . . 22
User has a number but is not enterprise-voice-enabled . . . 22
Policies show "Global" instead of the named policy . . . 22
Extension or local-format dialing fails . . . 23
Emergency call has no location or no notification . . . 23
Poor call quality / one-way or no audio . . . 23
Auto attendant or call queue doesn't answer / routes wrong . . . 23
Operator Connect numbers never appear . . . 24
Government cloud (GCC High / DoD) behaves differently . . . 24
Rollback needed for a user . . . 24
7. Best Practices — Teams Phone (Calling Plans & Operator Connect) 24
Choose the PSTN path deliberately, per number . . . 24
Treat emergency calling as first-class, not a footnote . . . 25
Pilot first, always, with a rollback . . . 25
Standardize and reuse a small set of named artifacts . . . 25
Manage numbers as inventory . . . 26
Right-size licensing and voice apps . . . 26
Design for call quality and support . . . 26
Automate safely . . . 26

Mind the cloud and the boundary 27
8. Deliverables Summary — Teams Phone (Calling Plans & Operator Connect) 27
Documentation 27
PowerShell scripts (5) — MicrosoftTeams module; read-only or `-WhatIf` by default; no stored credentials 27
Appendix A — PowerShell Scripts 29

Executive Summary — Microsoft Teams Phone Enablement (Calling Plans & Operator Connect)

Outcome

Enterprise voice runs in **Microsoft Teams Phone**: the right PSTN path is chosen per number, phone numbers are assigned, dial plans and calling policies are in place, **emergency calling is configured and tested**, and auto attendants and call queues route callers correctly — so users get reliable inbound and outbound calling directly in Teams, with no separate telephony platform.

Approach

The solution enables Teams Phone via the **two non-SBC PSTN paths — Microsoft Calling Plans (Microsoft is the carrier) and Operator Connect (an integrated operator is the carrier)** — using only the Teams admin center, the Microsoft 365 admin center, and the MicrosoftTeams PowerShell module. It follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — enabling a small named pilot ring first, validating every gate (including a coordinated **live emergency-call test**), then expanding in waves under change control. Five PowerShell tools handle discovery, safe number/policy assignment, auto attendant/call queue build-out, validation, and scope/health — all read-only or `-WhatIf` by default, with no stored credentials. *(Direct Routing via a customer-run SBC is a distinct PSTN model and is delivered as a separate solution.)*

Why it's low-risk

- **Pilot-first with explicit pass/fail gates** — voice never goes wide before it is proven on a small ring, and a documented rollback (`Remove-CsPhoneNumberAssignment` + re-grant global policies) is ready at every step.
- **Emergency calling is treated first-class** — civic addresses and dynamic locations are registered *before* numbers are assigned, security-desk notifications are configured, and a live emergency test is a mandatory gate. (For these PSTN paths, PSAP routing is Microsoft's/the operator's responsibility — confirmed and tested with them, not assumed.)
- **Safe-by-default automation** — every write runs `-WhatIf` first; modern-auth sign-in; no hardcoded secrets; least-privilege roles.
- **Honest about boundaries** — plan-, country-, operator-, and cloud-dependent behavior (Calling Plan vs Operator Connect availability, pay-as-you-go billing, GCC High/DoD differences) is flagged "verify in your tenant," and legal/records determinations for emergency addresses are deferred to facilities/EHS/compliance. You get a proven method, delivered at a fraction of typical consulting cost.

1. Solution Overview — Microsoft Teams Phone Enablement (Calling Plans & Operator Connect)

What this solution is

This solution turns on **enterprise voice in Microsoft Teams Phone** using the **two non-SBC PSTN connectivity paths — Microsoft Calling Plans and Operator Connect**. It is a repeatable, pilot-first method to license users for Teams Phone, acquire and assign phone numbers, configure calling and emergency-calling policy, build auto attendants and call queues, and validate that inbound and outbound calls (including emergency calls) work — using only the **Microsoft Teams admin center** (`https://admin.teams.microsoft.com`), the **Microsoft 365 admin center**, and the **MicrosoftTeams PowerShell module** (with Microsoft Graph where noted). No third-party tooling is required.

> ***Scope boundary.*** *This solution covers the* ***two Microsoft-hosted / operator-hosted PSTN paths*** *only.* ***Direct Routing (bring-your-own-carrier via a certified Session Border Controller)*** *is a separate PSTN model with its own trunks, voice routes, PSTN usages, and SBC lifecycle, and is* ***out of scope here*** *— it is delivered as a separate solution. Where a control (e.g. an Online Voice Routing Policy or an Emergency Call Routing Policy) exists* ***only*** *for Direct Routing, this document says so and does not apply it.*

The three PSTN options (and why this solution uses two of them)

Teams Phone needs a PSTN (public switched telephone network) connection to place and receive external calls. Microsoft offers three ways to provide it:

PSTN path	Who provides the carrier service	Where numbers come from	In scope here
Microsoft Calling Plan	Microsoft is your telephony carrier	Microsoft (get new numbers or port in via the Teams admin center)	**Yes**
Operator Connect	A participating telecom operator you already use, integrated into Teams	The operator (provisioned into your tenant via the operator's portal)	**Yes**
Direct Routing	Your own carrier via an SBC you (or a partner) run	Your carrier	No — separate solution

- **Calling Plan** is the fastest path: Microsoft is the carrier, numbers and minutes are bought in

the admin center, and there is no infrastructure to run. Availability, number types, and the billing model (fixed calling plan vs pay-as-you-go) are **country/region- and plan-dependent — verify what is offered for your tenant and country in the Teams admin center**.

- **Operator Connect** keeps your existing operator relationship and contract while delivering calling

natively in Teams: the operator provisions numbers directly into your tenant and you assign them in Teams. Which operators are available is **region-dependent — verify in the Teams admin center (`Voice → Operators`) for your tenant**.

- Both paths share the **same downstream configuration** in Teams (licensing, number assignment,

dial plans, calling policy, emergency calling, auto attendants, call queues). This solution treats the choice of Calling Plan vs Operator Connect as a per-number decision and configures everything after that point identically.

How it works (end to end)

1. **License.** Each calling user needs a **Teams Phone** license, plus — for Calling Plans — a

Microsoft Calling Plan (or pay-as-you-go add-on). Operator Connect needs the Teams Phone license but **no** Microsoft Calling Plan (the operator bills the PSTN service). Resource accounts that back auto attendants and call queues need a **Teams Phone Resource Account** license. (Exact SKU names and the pay-as-you-go / Communications Credits model are **plan-dependent — verify current SKUs and billing in the M365 admin center**.)

1. **Acquire numbers.** For **Calling Plans**, get new numbers or submit a port order in the Teams

admin center (`Voice → Phone numbers`). For **Operator Connect**, the operator provisions numbers into the tenant (via the operator portal / `Voice → Operators`); they then appear in `Phone numbers` ready to assign.

1. **Assign numbers + enable voice.** Assign a number to each pilot user with

`Set-CsPhoneNumberAssignment` (specifying the correct `-PhoneNumberType` of `CallingPlan` or `OperatorConnect`), which also enables enterprise voice for the user.

1. **Normalize dialing.** Apply a **Tenant Dial Plan** so user-dialed strings (extensions, local

formats) normalize to E.164. (The Tenant Dial Plan applies to all PSTN paths; the **Online Voice Routing Policy** is Direct-Routing-only and is *not* used here.)

1. **Calling policy.** Grant a **Teams Calling Policy** that sets the calling features users get

(call forwarding, delegation, call park, busy-on-busy, etc.).

1. **Emergency calling.** Configure **civic addresses and dynamic emergency locations (LIS)** and a

Teams Emergency Calling Policy (security-desk notifications). For Calling Plans and Operator Connect, **emergency-call routing to the correct PSAP is handled by Microsoft / the operator** — the **Emergency Call *Routing* Policy** is a Direct-Routing control and is **not** used here.

1. **Voice apps.** Create **resource accounts**, then **auto attendants** and **call queues**, assign

service numbers to the resource accounts, and connect them.

1. **Validate.** Confirm each user is enterprise-voice-enabled with the right number, that calls flow

in and out, that the emergency address/notification is correct, and that auto attendants and queues answer and route.

1. **Expand.** After the pilot passes every gate, repeat number assignment and policy grants for the

broader population under change control.

Reference design

• **One pilot ring first.** A small named user set (`Teams Phone Pilot – Voice – Pilot`) is fully

enabled and validated — including a live emergency-call test coordinated with the carrier/operator — before any broad rollout.

• **A small number of reusable policies**, named once and reused: a standard calling policy, an emergency calling policy per notification scope, and a tenant dial plan per country/dialing habit.

• **Resource accounts back every voice app.** Auto attendants and call queues are always fronted by a licensed resource account holding the service number.

• **Emergency location is data-driven.** Civic addresses and network identifiers (subnets / Wi-Fi access points / switches / ports) are registered in the Location Information Service so a user's emergency location follows where they actually connect.

Benefits

• **The right PSTN path, deliberately chosen** — Calling Plan where speed and simplicity win, Operator Connect where an existing operator relationship and contract should be kept, without standing up SBC infrastructure.

• **Numbers assigned cleanly and reversibly**, with a full before/after inventory and safe, `-WhatIf`-gated automation.

• **Emergency calling configured correctly**, with dynamic locations and security-desk notifications — the highest-risk part of any voice rollout, treated first-class.

• **Auto attendants and call queues** that route callers to the right people with published business hours and holidays.

• **Pilot-first, with explicit pass/fail gates and a documented rollback**, so a voice change never goes wide before it is proven on a small ring.

Limitations (be honest with buyers)

• **Direct Routing is not covered** — if you must bring your own carrier via an SBC, use the separate Direct Routing solution.

• **Calling Plan and Operator Connect availability, number types, and pricing are country/region- and operator-dependent. **Toll-free numbers, international dialing, and pay-as-you-go vs fixed-plan billing vary —** verify what is available for your country and tenant** in the Teams admin center; this solution provides the method, not a guarantee of availability.

• **Government clouds differ.** In **GCC High / DoD**, Calling Plan and Operator Connect availability, admin portals, and some cmdlet behavior are **different from Commercial — verify in your cloud/tenant** before assuming a feature or URL applies.

• **Emergency-call *routing* for Calling Plans/Operator Connect is Microsoft's / the operator's responsibility. **This solution configures locations and notifications, but the actual PSAP routing and any test-call capability depend on the carrier/operator —** coordinate a validated emergency test with

them**, do not assume 911/112 works until tested.

- **Number porting takes time** and is carrier-driven; port orders can take days to weeks and require accurate account/billing details. Plan cutover accordingly.

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and admin center. The standard set:

Artifact	Name
Number/license inventory export	`Number Inventory – Teams Phone – Discovery`
Pilot user group	`Teams Phone Pilot – Voice – Pilot`
Tenant dial plan	`Dial Plan – US – Prod`
Calling policy	`Calling Policy – Standard – Prod`
Emergency calling policy	`Emergency Calling Policy – HQ – Prod`
Emergency civic address	`Emergency Address – HQ`
Emergency dynamic location	`Emergency Location – HQ-Floor1`
Resource account (auto attendant)	`Resource Account – AA-MainLine`
Resource account (call queue)	`Resource Account – CQ-Support`
Auto attendant	`Auto Attendant – MainLine – Prod`
Call queue	`Call Queue – Support – Prod`
Validation report export	`Voice Validation – Teams Phone – Report`

The standard deployment model

> *Assess readiness and choose the PSTN path per number → design the policies, dial plan, emergency locations, and voice apps → configure them → enable a small **pilot** ring → validate every gate (including a live emergency test) → expand to the broader population under change control, with a documented rollback at every step. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements — Microsoft Teams Phone (Calling Plans & Operator Connect)

These are the **solution-specific** prerequisites for enabling Teams Phone via Calling Plans and Operator Connect. They build **on top of** the shared **Global Requirements — Microsoft Teams Solutions** (`global-requirements-teams.md`), which already cover: a licensed Microsoft 365 + Teams tenant, least-privilege Teams admin roles, the MicrosoftTeams PowerShell + Graph toolchain, an identity/group source of truth, a pilot scope + change control, and security/compliance stakeholders. **Verify the six global requirements first**, then confirm each item below. Each requirement lists **Description / Why /**

Verify / Implement if missing.

2.1 Teams Phone licensing for every calling user

• **Description:** Each user who will make/receive PSTN calls needs a **Teams Phone** license (Teams Phone Standard, or the equivalent included in an eligible suite/add-on). Resource accounts that back auto attendants and call queues need a **Microsoft Teams Phone Resource Account** license.

• **Why required:** Enterprise voice, number assignment, and voice apps are gated on Teams Phone licensing; without it `Set-CsPhoneNumberAssignment` and voice-policy grants will not apply.

• **How to verify:** M365 admin center → `Billing → Licenses`; or per user `Get-CsOnlineUser -Identity <upn> | Select-Object UserPrincipalName,FeatureTypes` and confirm the Teams Phone service plan is assigned. (Exact SKU names are **plan-dependent — verify in your tenant**.)

• **How to implement if missing:** Purchase/assign the Teams Phone license (and Resource Account licenses) before assigning numbers. Prefer **group-based licensing** for the pilot group so scope is managed by membership.

2.2 A chosen PSTN path per number — Calling Plan and/or Operator Connect

• **Description:** For each number/user, decide the PSTN path: **Microsoft Calling Plan** (Microsoft is the carrier — requires a Calling Plan or pay-as-you-go add-on per user) **or Operator Connect** (an integrated operator is the carrier — no Microsoft Calling Plan needed).

• **Why required:** The path determines what you must buy, where numbers come from, and the `-PhoneNumberType` used at assignment (`CallingPlan` vs `OperatorConnect`).

• **How to verify:** Teams admin center → `Voice → Calling plans` (confirm plans available for your country) and `Voice → Operators` (confirm a participating operator is enabled). Availability is **country/region- and operator-dependent — verify in your tenant**.

• **How to implement if missing:** For Calling Plans, buy the plan/add-on for the pilot users. For Operator Connect, select and enable a participating operator in `Voice → Operators` and complete the operator's onboarding so numbers can be provisioned into the tenant.

2.3 Phone numbers available to assign

• **Description:** A supply of **user (subscriber) numbers** for people and **service numbers** for auto attendants/call queues, obtained via the correct path.

• **Why required:** A user cannot be voice-enabled without a number; a voice app cannot receive external calls without a service number.

• **How to verify:** Teams admin center → `Voice → Phone numbers`; or

`Get-CsPhoneNumberAssignment -CapabilitiesContain UserAssignment` (and `-PstnAssignmentStatus Unassigned` to see spare inventory). Number **type/capability matters** — user vs service, calling-plan vs operator-connect.

- **How to implement if missing: Calling Plans** — `Voice → Phone numbers → Get new numbers`

(choose user or service subtype) or submit a **port order** (`Voice → Phone numbers → Port`) to move existing numbers. **Operator Connect** — request numbers from the operator; they appear in `Phone numbers` once provisioned. Port orders are carrier-driven and can take **days to weeks** — plan cutover timing.

2.4 Emergency addresses and dynamic-location data (LIS)

- **Description:** Registered **civic (emergency) addresses**, and — for dynamic emergency calling —

network identifiers (subnets, Wi-Fi access points, network switches, and ports) mapped to locations in the **Location Information Service (LIS)**.

- **Why required:** Emergency calls must present a **dispatchable location**. A static address can be

attached to a number; dynamic emergency calling resolves a user's location from where they connect. This is the **highest-risk** part of the rollout.

- **How to verify:** Teams admin center → `Locations → Emergency addresses`; or

`Get-CsOnlineLisCivicAddress` and `Get-CsOnlineLisLocation`. For dynamic mapping, confirm subnets/APs are registered (`Get-CsOnlineLisSubnet`, `Get-CsOnlineLisWirelessAccessPoint`).

- **How to implement if missing:** Register validated civic addresses (`New-CsOnlineLisCivicAddress`),

create places/locations within them (`New-CsOnlineLisLocation`), and map network identifiers (`Set-CsOnlineLisSubnet`, `Set-CsOnlineLisWirelessAccessPoint`, `Set-CsOnlineLisSwitch`, `Set-CsOnlineLisPort`). **The determination of which addresses are legally required and validated is deferred to your facilities/EHS/compliance function** — this solution provides the technical method.

2.5 Emergency-call routing responsibility confirmed (carrier/operator)

- **Description:** For Calling Plans and Operator Connect, **emergency calls are routed to the correct

PSAP by Microsoft / the operator**, not by a customer-defined routing policy. Confirm the carrier/operator's emergency-calling support for each country and any** validated test-call** method.

- **Why required:** The customer-side **Emergency Call *Routing* Policy** is a **Direct-Routing**

control and does **not** apply here; assuming otherwise leaves a gap. You must know how emergency routing works for your path and how to safely test it.

- **How to verify:** Review Microsoft's Calling Plan emergency-calling documentation for your country

and the operator's emergency-calling terms; confirm whether a test-call service exists. Emergency behavior is **country/plan-dependent — verify in your tenant/region**.

- **How to implement if missing:** Engage Microsoft/the operator to confirm emergency support and a

test method **before** going live; schedule the validated emergency test as a pilot gate.

2.6 Admin roles for voice, numbers, and emergency configuration

• **Description:** Beyond the global least-privilege Teams roles, confirm the operator has the rights to assign numbers, grant voice policies, and manage LIS/emergency data — typically **Teams Administrator** (or **Teams Communications Administrator** for voice-scoped work).

• **Why required:** Number assignment, dial plans, calling/emergency policy, and LIS changes require the corresponding Teams admin role.

• **How to verify:** M365 admin center / Entra roles → confirm the role; test with a read-only `Get-CsPhoneNumberAssignment` and `Get-CsTeamsCallingPolicy`.

• **How to implement if missing:** Have a Global Administrator assign the least-privilege Teams role under change control.

2.7 Network readiness for voice quality (and dynamic location)

• **Description:** A network that supports real-time media (adequate bandwidth, low latency/jitter, and the required Microsoft 365 network endpoints reachable), and — for dynamic emergency calling — a documented mapping of sites to subnets/APs/switches.

• **Why required:** Call quality and reliable dynamic-location resolution depend on the network; poor media paths cause dropped/garbled calls regardless of licensing.

• **How to verify:** Confirm the Teams media/optimize network endpoints are allowed and QoS (if used) is in place; validate a sample site's subnet/AP inventory against LIS.

• **How to implement if missing:** Align the network to Microsoft's Teams network guidance and capture the site-to-network-identifier mapping needed for §2.4 before enabling users at that site.

2.8 Resource-account service numbers for voice apps (if AAs/CQs are in scope)

• **Description:** For auto attendants and call queues that answer external calls, a **service number** (toll or toll-free) associated with a licensed resource account.

• **Why required:** An auto attendant or call queue only receives external calls through a resource account that holds a service number. Toll-free service numbers may require **Communications Credits / pay-as-you-go** — **verify the current billing model in your tenant**.

• **How to verify:** `Voice → Phone numbers` filtered to service numbers; per app instance `Get-CsOnlineApplicationInstance`.

• **How to implement if missing:** Acquire service numbers (Calling Plan `Get new numbers` service

subtype, or from the operator), create resource accounts, license them (§2.1), and assign the number.

*The deployment model is the shared Teams model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — with explicit pass/fail gates and a documented rollback for production voice changes. Automation is delivered as **PowerShell** (MicrosoftTeams module / Graph) with **no hardcoded credentials or secrets**.*

Global Requirements — Microsoft Teams Solutions

These requirements apply to **every** solution in the **Teams** category (Teams governance, meetings, external access, Teams Phone, apps, Teams Rooms, and Teams security/compliance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Teams solution).*** *The **Teams admin center** (admin.teams.microsoft.com), the **Microsoft 365 admin center**, the **MicrosoftTeams PowerShell module**, and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Where a capability, policy, or portal path is **plan- or version-dependent** (Teams Phone licensing, Calling Plans vs Operator Connect vs Direct Routing, Premium features, Commercial vs Government cloud), it is **cited or flagged "verify in your tenant"** — never invented. **Third-party devices and apps** (certified Teams Rooms hardware, SBCs, ISV apps) are integrated at the pattern level; their internals are deferred to the vendor's own docs. Automation is **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Microsoft Teams and appropriate licensing

- **Description:** A Microsoft 365 tenant with **Microsoft Teams** enabled and the licensing the capability requires (e.g. Teams Phone + a calling plan/Operator Connect for voice, Teams Premium for premium features, the appropriate E-plan for advanced compliance).
- **Why required:** All policy, voice, app, and compliance configuration depends on the right plan.
- **How to verify:** In the M365 admin center confirm the Teams subscription and relevant add-ons.
- **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Administrative access (least-privilege Teams roles)

- **Description:** Access with the least-privilege role the solution names — typically **Teams Administrator **(or Teams Communications/Device Administrator), with** Global Administrator** only where genuinely required.
- **Why required:** Teams policy, voice, app, and org-wide settings require the corresponding role.
- **How to verify:** Confirm the role in the M365 admin center / Entra roles.
- **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

3. Microsoft Teams PowerShell + Graph toolchain

• **Description:** A workstation with the **MicrosoftTeams** PowerShell module (and Graph PowerShell where used), modern-auth / MFA-capable admin sign-in.

• **Why required:** Configuration, automation, and the provided scripts run through Teams PowerShell / Graph.

• **How to verify:** `Connect-MicrosoftTeams` succeeds; `Get-Module MicrosoftTeams`.

• **How to implement if missing:** `Install-Module MicrosoftTeams`; confirm modern-auth sign-in.

4. Identity, groups & lifecycle source of truth

• **Description:** A defined identity/group source (Entra ID groups, and where used **Microsoft 365 Groups **which back teams) and the** lifecycle** stance (creation, naming, expiration/retention) the org expects.

• **Why required:** Teams are backed by M365 Groups; governance, provisioning, and access key off groups and the lifecycle model.

• **How to verify:** Confirm which groups/OU model is authoritative and the lifecycle expectations.

• **How to implement if missing:** Define the group/lifecycle model before configuring governance.

5. A pilot scope and change control

• **Description:** A **pilot team / policy group / user set** to validate on before org-wide rollout, and a **change-control** process for production policy/voice/compliance changes.

• **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model; policy and voice changes are high-impact.

• **How to verify:** Confirm a named pilot scope and an approval/change window are available.

• **How to implement if missing:** Create a pilot scope and agree change control before production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security, compliance/records, and collaboration-governance** stakeholders whose policies the configuration must satisfy (external access, information barriers, retention, DLP, app governance).

• **Why required:** Collaboration governance and compliance settings must align to enterprise policy.

• **How to verify:** Confirm the applicable policies and named approvers.

• **How to implement if missing:** Identify the owners and capture requirements before configuring governance.

*The deployment model across every Teams solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production

changes. Automation is delivered as **PowerShell** (MicrosoftTeams module / Graph) — never with hardcoded credentials or secrets in code.*

3. Implementation Guide — Teams Phone (Calling Plans & Operator Connect)

Follow these steps **in order**. Configuration is done in the **Teams admin center** (`https://admin.teams.microsoft.com`) and the **MicrosoftTeams PowerShell module**; all write automation runs `-WhatIf` **first** (the scripts default to a dry run). Artifact names follow the `<Purpose> – <Scope> – <Stage>` convention from the Overview and are **reused verbatim** in every later step, test, and script.

> ***Model:*** *Assess readiness and choose the PSTN path → design the policies/dial plan/emergency locations/voice apps → configure them → enable a small **pilot** ring → validate every gate (including a live emergency test) → expand under change control, with rollback at each step. Pattern: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***
>
> ***Connect first (every step that uses PowerShell):*** `Connect-MicrosoftTeams` *— modern auth / MFA. No stored credentials. Confirm with* `Get-CsTenant | Select-Object DisplayName,TenantId`.

Step 1 — Assess readiness and inventory (Assess)

1. Run `TEAMSPHONE-Discovery.ps1` (read-only) to capture the baseline: which users are already

enterprise-voice-enabled, current number assignments, assigned voice policies, spare number inventory, and licensing signals. It writes the export `Number Inventory – Teams Phone – Discovery`.

1. In the Teams admin center, confirm your PSTN availability: `Voice → Calling plans` (Calling Plan

availability for your country) and `Voice → Operators` (a participating Operator Connect operator). Record, per pilot user/number, the chosen path (**Calling Plan** or **Operator Connect**) — this drives the `-PhoneNumberType` used later.

1. Confirm Teams Phone licensing on the pilot users (Requirement 2.1). Do not proceed for a user whose

license/plan is missing.

Gate: the inventory is captured, the PSTN path is decided per pilot number, and pilot users are correctly licensed.

Step 2 — Define the pilot scope (Design)

1. Create/confirm the pilot user group `Teams Phone Pilot – Voice – Pilot` (an Entra security group).

Keep it small (e.g. 5–15 users across one site) and include at least one user who can safely participate in a coordinated **emergency test call**.

1. Run `TEAMSPHONE-Scope.ps1` (read-only) against the group to list members, their licensing, and current voice state — confirming everyone is ready to enable.

Gate: `Teams Phone Pilot – Voice – Pilot` exists, is populated, and every member is licensed.

Step 3 — Acquire numbers (Design/Configure)

1. **Calling Plans:** Teams admin center → `Voice → Phone numbers → Get new numbers` (choose the **user (subscriber)** subtype for people, **service** subtype for auto attendants/queues), or submit a **port order** (`Voice → Phone numbers → Port`) to move existing numbers. Port orders are carrier-driven — **allow days to weeks**.
1. **Operator Connect:** request numbers from the operator (via the operator's portal / `Voice → Operators`); they appear under Phone numbers` once provisioned into the tenant.
1. Re-run `TEAMSPHONE-Discovery.ps1` to confirm the new numbers show as **Unassigned** and of the expected type/capability.

Gate: enough user numbers for the pilot and any service numbers for voice apps are present and unassigned.

Step 4 — Register emergency addresses and dynamic locations (Configure — do this BEFORE assigning numbers)

1. In `Locations → Emergency addresses`, register and validate the site's civic address as `Emergency Address – HQ` (or via `New-CsOnlineLisCivicAddress`). Address validation is country-dependent — **verify in your tenant**.
1. Create the place(s) within it as `Emergency Location – HQ-Floor1` (`New-CsOnlineLisLocation`).
2. Map network identifiers to that location so location is **dynamic**: subnets (`Set-CsOnlineLisSubnet`), Wi-Fi access points (`Set-CsOnlineLisWirelessAccessPoint`), and — for wired — switches/ports (`Set-CsOnlineLisSwitch`, `Set-CsOnlineLisPort`).
1. **The choice of which addresses are legally required and validated is deferred to facilities/EHS/compliance** (Requirement 2.4) — you implement what they specify.

Gate: `Emergency Address – HQ` is validated and `Emergency Location – HQ-Floor1` is mapped to the site's network identifiers.

Step 5 — Build the calling policy, dial plan, and emergency calling policy (Configure)

1. **Calling policy** — create `Calling Policy – Standard – Prod`

(`New-CsTeamsCallingPolicy -Identity "Calling Policy - Standard - Prod" ...`) with the calling features the org allows (call forwarding, delegation, call park, busy-on-busy, etc.). *(PowerShell identities cannot contain the en-dash; use the ASCII form `Calling Policy - Standard - Prod` in cmdlets and the display name `Calling Policy – Standard – Prod` in the admin center — treat them as the same artifact.)*

1. **Tenant dial plan** — create `Dial Plan - US - Prod`

(`New-CsTenantDialPlan`) and add normalization rules (`New-CsVoiceNormalizationRule`) so extension and local-format dialing normalize to E.164. *(The Tenant Dial Plan applies to all PSTN paths; the **Online Voice Routing Policy** is Direct-Routing-only and is NOT created here.)*

1. **Emergency calling policy** — create `Emergency Calling Policy - HQ - Prod`

(`New-CsTeamsEmergencyCallingPolicy`) to define **security-desk notification** (who is notified when an emergency call is placed, and the notification mode). *(Do NOT create an Emergency Call **Routing** Policy — that is a Direct-Routing control; for Calling Plans/Operator Connect, routing to the PSAP is handled by Microsoft/the operator, Requirement 2.5.)*

Gate: the three artifacts exist with agreed settings and correct names.

Step 6 — Dry-run the pilot enablement (Pilot — no changes yet)

1. Run `TEAMSPHONE-Set-NumbersAndPolicy.ps1 -WhatIf` (the default) with the pilot group, the

number-to-user mapping, the chosen `-PhoneNumberType`, and the policy/dial-plan/emergency-location names. With `-WhatIf` it makes **no changes** — it prints, per user, exactly which number it would assign, which policies it would grant, and which emergency location it would attach.

1. Read every line. Confirm each user, number, `PhoneNumberType`, dial plan, calling policy, emergency

calling policy, and location is correct and that no production/non-pilot user appears.

Gate: the dry run shows exactly the intended changes for pilot users only, with no errors.

Step 7 — Enable the pilot (Pilot)

1. Re-run `TEAMSPHONE-Set-NumbersAndPolicy.ps1 -Execute` for `Teams Phone Pilot – Voice – Pilot`.

For each user it:

• assigns the number and location with

`Set-CsPhoneNumberAssignment -Identity <upn> -PhoneNumber <+E164> -PhoneNumberType <CallingPlan|OperatorConnect> -LocationId <id>` (this also enables enterprise voice),

• grants `Grant-CsTenantDialPlan`, `Grant-CsTeamsCallingPolicy`, and

`Grant-CsTeamsEmergencyCallingPolicy` with the named artifacts.

1. Allow policy propagation (assignments can take time to fully apply) before testing.

Gate: every pilot user is enterprise-voice-enabled with the correct number and named policies.

Step 8 — Build auto attendants and call queues (Configure/Pilot)

1. Create resource accounts `Resource Account – AA-MainLine` and `Resource Account – CQ-Support`

(`New-CsOnlineApplicationInstance`), license them with a **Teams Phone Resource Account** license (Requirement 2.1), then assign each a **service number** with `Set-CsPhoneNumberAssignment -Identity <resourceaccount> -PhoneNumber <+E164> -PhoneNumberType <path>`.

1. Run `TEAMSPHONE-Set-VoiceApps.ps1 -WhatIf` to preview creation of:

• `Call Queue – Support – Prod` (`New-CsCallQueue`) — agents, routing method, overflow/timeout,

backed by `Resource Account – CQ-Support`.

• `Auto Attendant – MainLine – Prod` (`New-CsAutoAttendant`) — business-hours/holiday schedule

(`New-CsOnlineTimeRange`, `New-CsOnlineSchedule`), menu and options (`New-CsAutoAttendantMenu`, `New-CsAutoAttendantMenuOption`, `New-CsAutoAttendantCallableEntity`) routing to the call queue, backed by `Resource Account – AA-MainLine`.

1. Re-run with `-Execute` to create them, then associate each resource account to its app

(`New-CsOnlineApplicationInstanceAssociation`).

Gate: calling the auto attendant's number reaches the menu; the menu routes to the call queue, which rings the agents.

Step 9 — Validate the pilot (Validate)

1. Run `TEAMSPHONE-Validation.ps1` to produce `Voice Validation – Teams Phone – Report` — confirm,

per pilot user: enterprise-voice enabled, correct `LineURI`/number, correct dial plan and calling policy, and emergency location attached.

1. Perform the live call tests in the **Testing Guide** (§5): outbound PSTN, inbound PSTN, and a

coordinated emergency test call (Requirement 2.5) confirming the security-desk notification fires and the presented location is correct.

Gate: every test in §5 passes, including the emergency test. Do not expand until this gate is met.

Step 10 — Expand under change control (Expand)

1. Add the next wave of users to a broader scope, repeat Steps 6–7 with `-WhatIf` then `-Execute`,

reusing the same named policies and dial plan.

1. Re-run `TEAMSPHONE-Validation.ps1` after each wave to refresh `Voice Validation – Teams Phone –

Report` and catch any user that did not fully enable.

1. Register additional sites' emergency addresses/locations (Step 4) **before** enabling users at each new site.

Rollback (any step)

Voice changes are reversible. To back a user out: `Remove-CsPhoneNumberAssignment -Identity <upn> -PhoneNumber <+E164> -PhoneNumberType <path>` (unassigns the number and disables enterprise voice), and re-grant the prior/global policies (`Grant-CsTeamsCallingPolicy -Identity <upn> -PolicyName $null`, likewise dial plan and emergency calling policy → global). For voice apps, remove the association (`Remove-CsOnlineApplicationInstanceAssociation`) and unassign the service number. `TEAMSPHONE-Set-NumbersAndPolicy.ps1 -Rollback -WhatIf` previews the exact reversal per user before you apply it.

Order of operations

`Assess/inventory → pilot scope → acquire numbers → emergency addresses/locations → policies + dial plan → dry-run → enable pilot → auto attendants/queues → validate (incl. emergency test) → expand`

5. Testing & Validation — Teams Phone (Calling Plans & Operator Connect)

Run these after the pilot enablement (Implementation §7–§8) and after **each** expansion wave. Each test states the **procedure** and the **expected pass/fail** signal. The automated report `Voice Validation – Teams Phone – Report` (from `TEAMSPHONE-Validation.ps1`) backs Tests 1–4; the live call tests (5–8) must be performed by a person. **Do not expand past the pilot until every test passes — especially the emergency test (Test 7).**

Test 1 — User is enterprise-voice-enabled with the correct number

- **Procedure:** `Get-CsOnlineUser -Identity <pilot-upn> | Select-Object UserPrincipalName,EnterpriseVoiceEnabled,LineURI`

and `Get-CsPhoneNumberAssignment -AssignedPstnTargetId <pilot-upn>`.

- **Pass:** `EnterpriseVoiceEnabled = True`, `LineURI` shows the intended `tel:+E164` number, and the

number's `PhoneNumberType` matches the chosen path (`CallingPlan` or `OperatorConnect`).

- **Fail:** `EnterpriseVoiceEnabled = False`, no/incorrect `LineURI`, or wrong number type → re-check

licensing (§2.1) and re-run `TEAMSPHONE-Set-NumbersAndPolicy.ps1 -Execute` for the user.

Test 2 — Voice policies and dial plan are assigned

- **Procedure:** `Get-CsOnlineUser -Identity <pilot-upn> | Select-Object TeamsCallingPolicy,TenantDialPlan,TeamsEmergencyCallingPolicy`.

• **Pass:** `TeamsCallingPolicy = Calling Policy - Standard - Prod`, `TenantDialPlan = Dial Plan - US -

Prod`, `TeamsEmergencyCallingPolicy = Emergency Calling Policy - HQ - Prod`.

• **Fail:** any shows global/blank when it should be the named policy → re-grant with the correct

`Grant-Cs*` cmdlet and confirm the artifact name matches exactly.

Test 3 — Emergency location is attached

• **Procedure:** `Get-CsPhoneNumberAssignment -AssignedPstnTargetId <pilot-upn> | Select-Object TelephoneNumber,LocationId`

and cross-check with `Get-CsOnlineLisLocation -LocationId <id>`.

• **Pass:** the number has a `LocationId` resolving to `Emergency Location – HQ-Floor1` (or the correct

site place), and dynamic identifiers (subnet/AP) for the pilot site are registered.

• **Fail:** `LocationId` empty or wrong → attach the location via

`Set-CsPhoneNumberAssignment -LocationId <id>` and confirm the LIS mapping (§ Implementation Step 4).

Test 4 — Spare inventory and no double-assignment

• **Procedure:** `TEAMSPHONE-Discovery.ps1` (or `Get-CsPhoneNumberAssignment -PstnAssignmentStatus

Assigned`).

• **Pass:** each assigned number maps to exactly one target; no number appears assigned to two objects;

spare numbers still show `Unassigned`.

• **Fail:** a duplicate/orphaned assignment → unassign the incorrect one with

`Remove-CsPhoneNumberAssignment` and reassign correctly.

Test 5 — Outbound PSTN call

• **Procedure:** From a pilot user's Teams client, place a call to an external mobile. Try an extension

or local-format number to confirm the dial plan normalizes it.

• **Pass:** the call connects, two-way audio is clear, and the called party sees the pilot user's

assigned number as caller ID; local/extension dialing normalizes correctly.

• **Fail:** call fails, one-way/no audio, or normalization wrong → check network readiness (§2.7),

`TenantDialPlan` normalization rules, and number assignment.

Test 6 — Inbound PSTN call

- **Procedure:** From an external phone, call the pilot user's assigned number.
- **Pass:** the pilot user's Teams client rings and answers with clear two-way audio.
- **Fail:** no ring / straight to voicemail unexpectedly → confirm the number is assigned to the user

(Test 1), the user is signed in, and call-answering/forwarding settings are as expected.

Test 7 — Emergency call (coordinated, live) — MANDATORY GATE

- **Procedure: Coordinate with the carrier/operator and your security/facilities function first**

(Requirement 2.5). Using the method they confirm (a validated emergency test service where offered, or a supervised live test), place an emergency call from a pilot user at the pilot site.

- **Pass:** the call routes correctly for the country/region; the **security-desk notification** defined

in `Emergency Calling Policy – HQ – Prod` fires to the notification target; and the **location presented/known** matches `Emergency Location – HQ-Floor1` (or the user's actual dynamic location).

- **Fail:** no notification, wrong/absent location, or routing failure → **stop the rollout**; fix the

LIS mapping, the emergency calling policy, and confirm carrier/operator emergency support before re-testing. Emergency routing behavior is **country/plan-dependent — verify in your tenant/region**.

Test 8 — Auto attendant and call queue

- **Procedure:** Call the service number for `Auto Attendant – MainLine – Prod`. Navigate the menu to

the option that routes to `Call Queue – Support – Prod`. Have an agent answer; also test the overflow/timeout path (e.g. no agent available).

- **Pass:** the auto attendant answers with the greeting/menu, honors business-hours vs after-hours

routing, and the selected option rings the call queue agents; overflow/timeout behaves as configured.

- **Fail:** no answer, wrong routing, or schedule ignored → confirm the resource accounts are licensed

and hold their service numbers, the app associations exist (`Get-CsOnlineApplicationInstanceAssociation`), and the schedule/menu configuration.

Pilot exit criteria (all must be true)

- Tests 1–4 pass for **every** pilot user in `Voice Validation – Teams Phone – Report`.
- Tests 5 and 6 (outbound + inbound) pass with clear two-way audio.
- **Test 7 (emergency) passes** with correct notification and location.
- Test 8 passes for the auto attendant and call queue (if in scope).
- No unresolved errors in the validation report. Only then run Implementation **Step 10 (Expand)**.

6. Troubleshooting — Teams Phone (Calling Plans & Operator Connect)

Common failure modes and how to resolve them. Use the read-only scripts (`TEAMSPHONE-Discovery.ps1`, `TEAMSPHONE-Validation.ps1`, `TEAMSPHONE-Scope.ps1`) to pinpoint the issue before changing anything, and reuse the named artifacts from the Overview.

A number won't assign (`Set-CsPhoneNumberAssignment` errors)

- **Symptom:** assignment fails with a licensing, number-type, or "not found" error.
- **Likely causes & fixes:**
- **Missing Teams Phone license** — confirm the user has Teams Phone (and, for Calling Plans, a calling plan / pay-as-you-go add-on). Fix per Requirement 2.1, then retry.
- **Wrong** `-PhoneNumberType` — the type must match how the number was provisioned (`CallingPlan` vs `OperatorConnect`). Check the number in `Get-CsPhoneNumberAssignment` and pass the matching type.
- **Number not in tenant / still provisioning** — for Operator Connect, the operator must finish provisioning before the number appears; re-check `Voice → Phone numbers`. For Calling Plans, confirm the `Get new numbers` order or port order completed.
- **Number already assigned** — a number can only target one object; unassign it first (`Remove-CsPhoneNumberAssignment`).

User has a number but is not enterprise-voice-enabled

- **Symptom:** `Get-CsOnlineUser` shows `LineURI` set but `EnterpriseVoiceEnabled = False`, or calls don't work.
- **Fix:** `Set-CsPhoneNumberAssignment` normally enables enterprise voice; if it didn't, confirm licensing propagated (new licenses can take time), re-run the assignment, and allow policy propagation before testing. Validate with `TEAMSPHONE-Validation.ps1`.

Policies show "Global" instead of the named policy

- **Symptom:** `TeamsCallingPolicy` / `TenantDialPlan` / `TeamsEmergencyCallingPolicy` is blank/Global after enablement.
- **Fix:** re-grant with the exact policy name (`Grant-CsTeamsCallingPolicy -PolicyName "Calling Policy - Standard - Prod"`, etc.). **Watch the naming caveat:** PowerShell identities use the ASCII hyphen

form (`Calling Policy - Standard - Prod`), not the en-dash used in the display name — a mismatch means "policy not found." Confirm the policy exists (`Get-CsTeamsCallingPolicy`).

Extension or local-format dialing fails

- **Symptom:** dialing a full E.164 number works, but extensions or local formats fail.
- **Fix:** the **Tenant Dial Plan** normalizes dialed strings. Confirm `Dial Plan - US - Prod` is

granted to the user and its normalization rules (`New-CsVoiceNormalizationRule`) cover the pattern. Test rules with `Get-CsEffectiveTenantDialPlan -Identity` <upn> and its test cmdlet (`Test-CsEffectiveTenantDialPlan`) — **verify exact cmdlet availability in your tenant**. Remember the **Online Voice Routing Policy is Direct-Routing-only** and is not the cause here.

Emergency call has no location or no notification

- **Symptom:** Test 7 fails — the emergency call presents no/wrong location, or the security desk isn't

notified.

- **Fix:**
- **Location:** confirm the number has a `LocationId` (`Get-CsPhoneNumberAssignment

-AssignedPstnTargetId <upn>) `resolving to` Emergency Location – HQ-Floor1, `and that the site's subnets/APs are mapped (`Get-CsOnlineLisSubnet, Get-CsOnlineLisWirelessAccessPoint`). Attach with` Set-CsPhoneNumberAssignment -LocationId <id>`.

- **Notification:** confirm `Emergency Calling Policy – HQ – Prod` has the notification target/mode

set and is granted to the user.

- **Routing:** for Calling Plans/Operator Connect, PSAP routing is Microsoft's/the operator's

responsibility (the Emergency Call *Routing* Policy is Direct-Routing-only). If routing itself fails, engage the carrier/operator — **do not** try to fix it with a customer routing policy. Emergency behavior is **country/plan-dependent — verify in your tenant/region**.

Poor call quality / one-way or no audio

- **Symptom:** calls connect but audio is choppy, delayed, or one-way.
- **Fix:** this is almost always a **network** issue, not licensing. Verify the required Microsoft 365

media endpoints are reachable and not proxied, adequate bandwidth and low jitter/latency, and QoS if used (Requirement 2.7). Test from a different network to isolate. Check the client's audio device.

Auto attendant or call queue doesn't answer / routes wrong

- **Symptom:** calling the service number fails, ignores the schedule, or doesn't reach agents.
- **Fix:**
- Confirm the **resource account is licensed** (Teams Phone Resource Account) and **holds a service

number** (`Get-CsOnlineApplicationInstance`).

- Confirm the **association** exists (`Get-CsOnlineApplicationInstanceAssociation -Identity <resourceaccount>) linking the account to Auto Attendant – MainLine – Prod / Call Queue – Support – Prod`.
- Confirm the **schedule/menu** (business hours, holidays, DTMF option) and that the call queue has online agents and the expected routing/overflow/timeout.

Operator Connect numbers never appear

- **Symptom:** the operator is selected but no numbers show under `Voice → Phone numbers`.
- **Fix:** Operator Connect requires the operator to **provision** numbers into your tenant after onboarding. Confirm the operator is enabled in `Voice → Operators`, that onboarding/consent completed, and follow up with the operator — provisioning is on their side, not a Teams misconfiguration.

Government cloud (GCC High / DoD) behaves differently

- **Symptom:** a portal path, feature, or cmdlet result differs from what's documented.
- **Fix:** Calling Plan / Operator Connect availability, admin portals, and some behavior differ in government clouds. **Verify the feature, URL, and cmdlet behavior for your specific cloud/tenant** — do not assume Commercial guidance applies unchanged.

Rollback needed for a user

- **Symptom:** a user must be backed out of Teams Phone.
- **Fix:** `TEAMSPHONE-Set-NumbersAndPolicy.ps1 -Rollback -WhatIf` to preview, then `-Rollback -Execute` to unassign the number and reset the dial plan / calling / emergency policies to Global. For voice apps, remove the association (`Remove-CsOnlineApplicationInstanceAssociation`) and unassign the service number.

If a symptom isn't listed, start with `TEAMSPHONE-Validation.ps1` to get the per-user PASS/FAIL, then work the failing check with the matching section above. Plan-, country-, operator-, and cloud-dependent behavior is always "verify in your tenant" — never assume.

7. Best Practices — Teams Phone (Calling Plans & Operator Connect)

Durable practices for a reliable Teams Phone rollout on the non-SBC PSTN paths.

Choose the PSTN path deliberately, per number

- Use **Calling Plans** when you want Microsoft as the carrier with the least infrastructure and fast

provisioning, and the plan is available for your country.

• Use **Operator Connect** when you want to keep an existing operator relationship/contract and have the operator manage PSTN, without running an SBC.

• You can mix both in one tenant. Record the path per number so `-PhoneNumberType` is always correct at assignment. **Availability and pricing are country/operator-dependent — verify in your tenant.**

Treat emergency calling as first-class, not a footnote

• Register and validate emergency **civic addresses** and build **dynamic locations (LIS)** *before* assigning numbers at a site — never enable users at a location with no dispatchable location data.

• Map every site's **subnets, Wi-Fi access points, and (for wired) switches/ports** to a location so a user's emergency location follows where they actually connect.

• Configure **security-desk notifications** in the emergency calling policy so someone on-site is alerted when an emergency call is placed.

• Remember: for Calling Plans/Operator Connect, **PSAP routing is Microsoft's/the operator's responsibility **— the Emergency Call *Routing* Policy is Direct-Routing-only. Confirm and** test** emergency behavior with the carrier/operator; treat the emergency test as a hard gate.

• **The legal/records determination of required addresses is deferred to facilities/EHS/compliance** — implement what they specify; you provide the technical method and evidence.

Pilot first, always, with a rollback

• Enable a small named ring (`Teams Phone Pilot – Voice – Pilot`) and validate every gate — including a live emergency test — before any broad wave.

• Keep the documented rollback ready (`Remove-CsPhoneNumberAssignment` + re-grant global policies) and practice it once in the pilot so cutover is low-risk.

• Expand in waves under change control, re-validating after each wave.

Standardize and reuse a small set of named artifacts

• One calling policy, one emergency calling policy per notification scope, one dial plan per country/dialing habit — named `<Purpose> – <Scope> – <Stage>` and reused verbatim across docs, scripts, and the admin center.

• Avoid per-user bespoke policies; assign the named policy to groups so scope is membership-driven and

auditable.

• **Note the naming caveat:** PowerShell policy *identities* cannot contain the en-dash; use the ASCII

- form in cmdlets (`Calling Policy - Standard - Prod`) and the en-dash display name in the admin center — treat them as the same artifact and keep them in lockstep.

Manage numbers as inventory

• Keep an authoritative export (`Number Inventory – Teams Phone – Discovery`) and reconcile it

regularly: no double-assignments, spare numbers tracked, service numbers separated from user numbers.

• Plan **port orders** early — they are carrier-driven and can take days to weeks; never schedule a

cutover assuming an instant port.

• Reclaim numbers from leavers promptly (`Remove-CsPhoneNumberAssignment`) so inventory stays clean.

Right-size licensing and voice apps

• Assign **Teams Phone** to users who actually need PSTN calling; use **Teams Phone Resource Account**

licenses for auto attendant/call queue resource accounts (do not consume user licenses for them).

• Front every auto attendant/call queue with a **licensed resource account** holding the service

number; keep resource-account naming (`Resource Account – AA-MainLine` / `– CQ-Support`) consistent.

• Confirm the current billing model for toll-free/pay-as-you-go (**Communications Credits /

pay-as-you-go is plan-dependent — verify in your tenant**).

Design for call quality and support

• Verify network readiness (bandwidth, latency/jitter, required Microsoft 365 endpoints, optional QoS)

before enabling users at a site — voice quality is a network property, not a licensing one.

• Give users a short "what changed" guide (their new number, how to dial internationally, how call

queues/attendants work) and a support path for the first weeks.

Automate safely

• Run every write with `-WhatIf` **first**; only `-Execute` after reviewing the dry run.

• Use **modern-auth** `Connect-MicrosoftTeams` (MFA) — **never** hardcode credentials or secrets in

scripts; scope automation accounts to the least-privilege Teams role.

• Keep the discovery/validation scripts read-only and run them on a schedule to catch drift (users who

lost a policy, orphaned numbers, missing emergency locations).

Mind the cloud and the boundary

• In **GCC High / DoD**, availability, portals, and some behavior differ from Commercial — **verify in your cloud/tenant** before assuming a feature or URL.

• Keep **Direct Routing** work in its own solution — do not mix SBC/voice-route/Emergency-Call-Routing configuration into this non-SBC path.

8. Deliverables Summary — Teams Phone (Calling Plans & Operator Connect)

Everything included in this solution, on one page.

Documentation

• **01 — Solution Overview:** what Teams Phone enablement via Calling Plans and Operator Connect is, the three PSTN options (and why Direct Routing is separate), how it works end to end, the reference design, benefits, limitations, the naming convention, and the deployment model.

• **02 — Requirements:** solution-specific prerequisites (Teams Phone licensing, chosen PSTN path, numbers, emergency addresses/LIS, emergency-routing responsibility, admin roles, network readiness, resource-account service numbers) — each with Description / Why / Verify / Implement — building on the shared Teams Global Requirements.

• **03 — Implementation Guide:** exact ordered steps — assess/inventory, pilot scope, acquire numbers, register emergency addresses/locations, build policies + dial plan, dry-run, enable the pilot, build auto attendants/call queues, validate (incl. emergency test), and expand — every artifact named, with pass/fail gates and a rollback.

• **05 — Testing & Validation:** eight procedures (voice enablement, policies, emergency location, inventory, outbound/inbound calls, the mandatory live emergency test, and auto attendant/call queue) each with an expected pass/fail signal and pilot exit criteria.

• **06 — Troubleshooting:** common failure modes (number won't assign, user not voice-enabled, no emergency location, dial-plan normalization, call quality, auto attendant/queue routing, Operator Connect provisioning, government-cloud differences) and their resolutions.

• **07 — Best Practices:** durable practices for PSTN-path choice, emergency-calling discipline, pilot-first rollout, artifact standardization, number inventory, licensing, call quality, and safe automation.

• **Executive Summary:** the outcome, approach, and why the rollout is low-risk.

PowerShell scripts (5) — MicrosoftTeams module; read-only or `-WhatIf` by default; no stored credentials

1. `TEAMSPHONE-Discovery.ps1` — *Readiness & inventory (read-only).* Reports enterprise-voice state,

current number assignments, assigned voice policies, spare number inventory, and licensing signals; writes `Number Inventory – Teams Phone – Discovery`. Makes no changes.

1. `TEAMSPHONE-Set-NumbersAndPolicy.ps1` — *Number-assignment + voice-policy helper (defaults to

`-WhatIf`).* From a user→number→path mapping, assigns numbers (`Set-CsPhoneNumberAssignment`), attaches the emergency location, and grants the named dial plan, calling policy, and emergency calling policy. `-Execute` applies; `-Rollback` reverses. Safe by default.

1. `TEAMSPHONE-Set-VoiceApps.ps1` — *Auto attendant / call queue config helper (defaults to

`-WhatIf`).* Creates/associates resource accounts, call queues (`New-CsCallQueue`), and auto attendants (`New-CsAutoAttendant`) with a business-hours schedule and menu routing to the queue. Previews by default; `-Execute` applies.

1. `TEAMSPHONE-Validation.ps1` — *Calling + emergency-calling validation report.* Confirms, per

user, enterprise-voice enablement, correct number/type, assigned policies, and attached emergency location; flags gaps. Writes `Voice Validation – Teams Phone – Report`. Read-only.

1. `TEAMSPHONE-Scope.ps1` — *Scope / health helper (read-only).* Lists a pilot/target group's

members with their licensing and voice-readiness state, and surfaces tenant PSTN/health signals so you know who is ready to enable before you touch anything.

> *Every script starts with comment-based help (`.SYNOPSIS` / `.DESCRIPTION` / `.EXAMPLE` / `.NOTES`), connects with modern-auth `Connect-MicrosoftTeams` (no stored credentials), and defaults to read-only or `-WhatIf`. Write actions require an explicit `-Execute`.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TEAMSPHONE-Discovery.ps1

Purpose: Builds the Teams Phone baseline: which users are enterprise-voice-enabled, their assigned numbers and number type (CallingPlan / OperatorConnect / DirectRouting), assigned voice policies (calling policy, tenant dial plan, emergency calling policy), and spare unassigned number inventory. Read-only — makes no changes. Writes the export "Number Inventory - Teams Phone - Discovery". HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TEAMSPHONE-Discovery.ps1 -CsvPath C:\Temp\TEAMSPHONE-Inventory.csv (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS A console summary of voice-enabled users and number inventory, plus an optional per-user CSV (UPN, EnterpriseVoiceEnabled, LineURI, PhoneNumberType, policies). No changes are made. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Read-only. No stored credentials. > [CmdletBinding()] param([string]$CsvPath, [string]$ReportName = 'Number Inventory - Teams Phone - Discovery')

How to run: powershell -ExecutionPolicy Bypass -File .\TEAMSPHONE-Discovery.ps1 -CsvPath C:\Temp\TEAMSPHONE-Inventory.csv (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS A console summary of voice-enabled users and number inventory, plus an optional per-user CSV (UPN, EnterpriseVoiceEnabled, LineURI, PhoneNumberType, policies). No changes are made. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Read-only. No stored credentials. > [CmdletBinding()] param([string]$CsvPath, [string]$ReportName = 'Number Inventory - Teams Phone - Discovery')

TEAMSPHONE-Scope.ps1

Purpose: Lists the members of a pilot/target group (e.g. "Teams Phone Pilot - Voice - Pilot") with their current voice-readiness — enterprise-voice state, assigned number, and assigned policies — so you know exactly who is ready to enable before touching anything. Also surfaces tenant-level PSTN scope signals: number inventory by type/status and the count of enterprise-voice-enabled users. Read-only — makes no changes. HOW TO RUN .\TEAMSPHONE-Scope.ps1 -GroupName "Teams Phone Pilot - Voice - Pilot" (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS A per-member readiness table (Ready / Needs-enable) and a tenant PSTN summary. Nothing changes. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Read-only. No stored credentials. > [CmdletBinding()] param([Parameter(Mandatory)][string]$GroupName, [string]$CsvPath)

How to run: .\TEAMSPHONE-Scope.ps1 -GroupName "Teams Phone Pilot - Voice - Pilot" (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS A per-member readiness table (Ready / Needs-enable) and a tenant PSTN summary. Nothing changes. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Read-only. No stored credentials. > [CmdletBinding()] param([Parameter(Mandatory)][string]$GroupName, [string]$CsvPath)

TEAMSPHONE-Set-NumbersAndPolicy.ps1

Purpose: Enables Teams Phone for a set of users from a mapping CSV: assigns each number with the correct PhoneNumberType (CallingPlan / OperatorConnect), attaches the emergency LocationId, and grants the named tenant dial plan, calling policy, and emergency calling policy. SAFE BY DEFAULT — runs in -WhatIf mode (prints intended changes, changes nothing) unless -Execute is supplied. -Rollback previews/applies the reversal (unassign number, reset policies to global). HOW TO RUN Dry-run (default, NO changes): .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv Apply: .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv -Execute Roll back (preview then apply): .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv -Rollback .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv -Rollback -Execute (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) MappingCsv columns: UserPrincipalName, PhoneNumber (+E164), PhoneNumberType (CallingPlan|OperatorConnect), LocationId (optional), DialPlan, CallingPolicy, EmergencyCallingPolicy EXPECTED RESULTS -WhatIf: one line per user describing the exact assignment + grants that WOULD be made; nothing changes. -Execute: numbers assigned and policies granted for the listed users only; a per-user result is printed. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Default mode makes NO changes. No stored credentials. > [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [switch]$Execute, # absent = -WhatIf (default, no changes) [switch]$Rollback, # reverse: unassign number, reset policies to Global [string]$ResultCsv)

How to run: Dry-run (default, NO changes): .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv Apply: .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv -Execute Roll back (preview then apply): .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv -Rollback .\TEAMSPHONE-Set-NumbersAndPolicy.ps1 -MappingCsv .\pilot-numbers.csv -Rollback -Execute (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) MappingCsv columns: UserPrincipalName, PhoneNumber (+E164), PhoneNumberType (CallingPlan|OperatorConnect), LocationId (optional), DialPlan, CallingPolicy, EmergencyCallingPolicy EXPECTED RESULTS -WhatIf: one line per user describing the exact assignment + grants that WOULD be made; nothing changes. -Execute: numbers assigned and policies granted for the listed users only; a per-user result is printed. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Default mode makes NO changes. No stored credentials. > [CmdletBinding()] param([Parameter(Mandatory)][string]$MappingCsv, [switch]$Execute, # absent = -WhatIf (default, no changes) [switch]$Rollback, # reverse: unassign number, reset policies to Global [string]$ResultCsv)

TEAMSPHONE-Set-VoiceApps.ps1

Purpose: Builds the voice-app layer: resource accounts ("Resource Account - AA-MainLine" / "Resource Account - CQ-Support"), a call queue ("Call Queue - Support - Prod") backed by the CQ resource account, and an auto attendant ("Auto Attendant - MainLine - Prod") with a business-hours schedule and a menu option routing callers to the call queue. SAFE BY DEFAULT — runs in -WhatIf (prints intended actions, changes nothing) unless -Execute is supplied. NOTE: resource accounts must be licensed (Teams Phone Resource Account) and assigned a service number (Set-CsPhoneNumberAssignment) — do that before -Execute or the association will fail. HOW TO RUN Dry-run (default, NO changes): .\TEAMSPHONE-Set-VoiceApps.ps1 -AaUpn aa-mainline@contoso.com -CqUpn cq-support@contoso.com -AgentUpns "a@contoso.com","b@contoso.com" Apply: .\TEAMSPHONE-Set-VoiceApps.ps1 -AaUpn aa-mainline@contoso.com -CqUpn cq-support@contoso.com -AgentUpns "a@contoso.com","b@contoso.com" -Execute (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA)

EXPECTED RESULTS -WhatIf: prints each resource account, call queue, and auto attendant it WOULD create/associate. -Execute: creates the objects and associations; prints the resulting ObjectIds. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Default mode makes NO changes. No stored credentials. contoso.com is a documentation placeholder — substitute your tenant's routing domain. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AaUpn, # auto attendant resource account UPN [Parameter(Mandatory)][string]$CqUpn, # call queue resource account UPN [string[]]$AgentUpns = @(), # call queue agents [string]$AaName = 'Auto Attendant - MainLine - Prod', [string]$CqName = 'Call Queue - Support - Prod', [switch]$Execute # absent = -WhatIf (default, no changes))

How to run: Dry-run (default, NO changes): .\TEAMSPHONE-Set-VoiceApps.ps1 -AaUpn aa-mainline@contoso.com -CqUpn cq-support@contoso.com -AgentUpns "a@contoso.com","b@contoso.com" Apply: .\TEAMSPHONE-Set-VoiceApps.ps1 -AaUpn aa-mainline@contoso.com -CqUpn cq-support@contoso.com -AgentUpns "a@contoso.com","b@contoso.com" -Execute (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS -WhatIf: prints each resource account, call queue, and auto attendant it WOULD create/associate. -Execute: creates the objects and associations; prints the resulting ObjectIds. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Default mode makes NO changes. No stored credentials. contoso.com is a documentation placeholder — substitute your tenant's routing domain. > [CmdletBinding()] param([Parameter(Mandatory)][string]$AaUpn, # auto attendant resource account UPN [Parameter(Mandatory)][string]$CqUpn, # call queue resource account UPN [string[]]$AgentUpns = @(), # call queue agents [string]$AaName = 'Auto Attendant - MainLine - Prod', [string]$CqName = 'Call Queue - Support - Prod', [switch]$Execute # absent = -WhatIf (default, no changes))

TEAMSPHONE-Validation.ps1

Purpose: For each supplied user, confirms Teams Phone is correctly enabled: enterprise voice on, a LineURI/number assigned with the expected PhoneNumberType, the named calling policy / tenant dial plan / emergency calling policy granted, and an emergency LocationId attached to the number. Emits a PASS/FAIL per check so gaps are obvious. Read-only — makes no changes. Writes "Voice Validation - Teams Phone - Report". HOW TO RUN .\TEAMSPHONE-Validation.ps1 -UserUpns "a@contoso.com","b@contoso.com" `-ExpectCallingPolicy "Calling Policy - Standard - Prod"` -ExpectDialPlan "Dial Plan - US - Prod" ` -ExpectEmergencyPolicy "Emergency Calling Policy - HQ - Prod" -CsvPath C:\Temp\voice-validation.csv (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS A per-user, per-check PASS/FAIL table and an optional CSV. Any FAIL points at the exact remediation. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Read-only. No stored credentials. contoso.com is a documentation placeholder — substitute your tenant's users. > [CmdletBinding()] param([Parameter(Mandatory)][string[]]$UserUpns, [string]$ExpectCallingPolicy = 'Calling Policy - Standard - Prod', [string]$ExpectDialPlan = 'Dial Plan - US - Prod', [string]$ExpectEmergencyPolicy = 'Emergency Calling Policy - HQ - Prod', [string]$CsvPath, [string]$ReportName = 'Voice Validation - Teams Phone - Report')

How to run: .\TEAMSPHONE-Validation.ps1 -UserUpns "a@contoso.com","b@contoso.com" `-ExpectCallingPolicy "Calling Policy - Standard - Prod"` -ExpectDialPlan "Dial Plan - US - Prod" ` -ExpectEmergencyPolicy "Emergency Calling Policy - HQ - Prod" -CsvPath C:\Temp\voice-validation.csv (PowerShell 5.1+/7+, MicrosoftTeams module; connect with Connect-MicrosoftTeams — modern auth / MFA) EXPECTED RESULTS A per-user, per-check

PASS/FAIL table and an optional CSV. Any FAIL points at the exact remediation. .NOTES Teams — Teams Phone Enablement (Calling Plans & Operator Connect). Read-only. No stored credentials. contoso.com is a documentation placeholder — substitute your tenant's users. > [CmdletBinding()] param([Parameter(Mandatory)][string[]]$UserUpns, [string]$ExpectCallingPolicy = 'Calling Policy - Standard - Prod', [string]$ExpectDialPlan = 'Dial Plan - US - Prod', [string]$ExpectEmergencyPolicy = 'Emergency Calling Policy - HQ - Prod', [string]$CsvPath, [string]$ReportName = 'Voice Validation - Teams Phone - Report')